

CLIPMIND AI

Video Summarization & Key Moments Detection Platform

An Intelligent Full-Stack AI Platform for Speech-to-Text Transcription, Dual-Tier Summarization, Algorithmic Key Moments Detection, and Role-Based Learning Analytics

PROJECT DOCUMENTATION

A Final Project Technical Report

Submitted by

R SUBATHRA

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me throughout the development of ClipMind AI. I am deeply thankful to my project guide for their invaluable guidance, continuous encouragement, and constructive feedback at every stage of this project.

I extend my heartfelt thanks to the Head of the Department and the faculty members of my institution for providing the necessary resources, technical support, and a conducive environment that made this work possible.

I am also grateful to Infosys for the opportunity to work on this project, which allowed me to apply and strengthen my knowledge of full-stack development and artificial intelligence in a practical, real-world context.

Finally, I wish to thank my family and friends for their constant encouragement, patience, and support, which motivated me to complete this project successfully.

Submitted by,

R SUBATHRA

Abstract

Video has rapidly evolved into the dominant medium for academic education, corporate training, technical presentations, and online content dissemination. However, the non-linear consumption of long-form video assets presents significant cognitive friction and temporal inefficiency. Learners and professionals routinely spend hours watching or scrubbing through multimedia files to extract only a fraction of relevant insights. Manual transcription, note-taking, and highlight extraction are labor-intensive, error-prone, and unscalable.

ClipMind AI is an intelligent, full-stack, AI-powered video summarization, speech transcription, key moments detection, and classroom analytics platform. Built on a modern decoupled architecture combining a React 19 Single Page Application (SPA), a high-performance asynchronous FastAPI backend, PostgreSQL relational storage, and local machine learning inference runtimes (Faster-Whisper int8 and Ollama-hosted Llama 3.2), ClipMind AI provides an end-to-end multimedia processing lifecycle. The system ingests local video uploads or remote YouTube URLs, extracts 16 kHz mono PCM audio streams, performs neural speech-to-text transcription with Voice Activity Detection (VAD), generates concurrent dual-tier summaries (executive digests and analytical bullet points), clusters algorithmic key moments with timestamp navigation, and provides multilingual keyword explanations in over 12 languages.

Furthermore, the platform integrates robust Role-Based Access Control (RBAC) across four distinct personas (Learner, Educator, Content Creator, Administrator), featuring dedicated classroom engagement analytics, automated study material generation, full administration governance consoles, and client-side compiled PDF/TXT reports. This documentation details the complete 27 structured sections forming the engineering foundation of ClipMind AI, accompanied by annotated visual screenshots captured from the production environment.

Contents

1	Introduction	1
1.1	Project Overview	1
1.2	Background	1
1.3	Need for the Project	1
1.4	Purpose of the System	2
2	Problem Statement	2
3	Existing System Analysis	3
3.1	Existing Approach	3
3.2	Limitations of Existing System	3
4	Proposed System	4
4.1	Proposed Solution	4
4.2	Advantages of Proposed System	4
5	Objectives	5
6	Scope of the Project	6
7	System Requirements	6
7.1	Hardware Requirements	6
7.2	Software Requirements	6
8	Technology Stack	6
8.1	Frontend Technologies	6
8.2	Backend Technologies	7
8.3	Database Technologies	7
8.4	AI/ML Technologies	8
8.5	Video Processing Technologies	8
8.6	Authentication Technologies	8
8.7	Reporting Technologies	8
8.8	Deployment Technologies	8
9	System Architecture	8
10	System Workflow	9
11	System Modules	10
11.1	Authentication Module	10
11.2	Video Upload Module	11

11.3 Speech-to-Text Module	12
11.4 AI Summary Module	13
11.5 Key Moments Module	14
11.6 Keyword Explanation Module	15
11.7 Learning Material Module	15
11.8 Analytics Module	16
11.9 Report Generation Module	17
11.10 Administration Module	18
12 User Roles and Functionalities	19
12.1 Learner	19
12.2 Educator	20
12.3 Content Creator	20
12.4 Administrator	21
13 AI Processing Pipeline	21
14 Database Design	21
14.1 User Table (users)	21
14.2 Video Table (videos)	21
14.3 Database Relationships	23
15 Frontend Design	23
15.1 React Application	23
15.2 Page Structure	23
15.3 Component Structure	24
15.4 Navigation	24
15.5 API Communication	24
15.6 Analytics Visualization	24
16 Backend Design	24
16.1 FastAPI Application	24
16.2 API Routers	24
16.3 Database Integration	25
16.4 Authentication APIs	25
16.5 AI Processing APIs	25
17 API Endpoints	25
17.1 Authentication APIs	25
17.2 Video APIs	25
17.3 Transcript APIs	26

17.4 Summary APIs	26
17.5 Key Moments APIs	26
17.6 Analytics APIs	26
17.7 Learning Material APIs	26
17.8 Administration APIs	26
18 Security	27
18.1 Password Hashing	27
18.2 JWT Authentication	27
18.3 Role-Based Access	27
18.4 API Validation	27
19 Docker Implementation	27
19.1 Docker Architecture	27
19.2 Backend Dockerfile	27
19.3 Frontend Dockerfile	28
19.4 Docker Compose	28
19.5 Container Communication	28
19.6 Deployment Process	28
20 User Interface Screens	29
21 Testing	30
21.1 Authentication Testing	30
21.2 Video Upload Testing	30
21.3 Transcription Testing	30
21.4 Summary Testing	30
21.5 Key Moments Testing	30
21.6 Analytics Testing	30
21.7 Role-Based Testing	31
21.8 Docker Testing	31
22 Results and Discussion	31
23 Advantages	31
24 Conclusion	32
25 References	32

List of Figures

1	Authentication Module Interfaces	10
2	Upload Center and AI Processing Workflow Visualizer	11
3	AI Transcript Interface with Inline Editing and Detected Keyword Chips	13
4	AI Summary Generator with Quality Evaluation and Sharing Controls .	14
5	Key Moments Detection with Timestamps, Highlights, and Topics	15
6	Learning Materials Studio: Concepts, Notes, and Practice Quizzes	16
7	Classroom Content Analytics Dashboard with Engagement Trends	17
8	Client-Side Compiled Multi-Page PDF Analytics Report	18
9	Platform Administration Governance Consoles	19
10	Learner Role Workspaces	19
11	Educator Workstation Dashboard	20
12	Content Creator Role Workspaces	20
13	Administrator Governance Workspaces	21
14	Personal User Features and Account Views	30

List of Tables

1	Comparative Matrix: Traditional Systems vs. ClipMind AI	4
2	System Hardware Requirements	6
3	System Software Requirements	7
4	Database Schema: users Table	22
5	Database Schema: videos Table	22

1 Introduction

1.1 Project Overview

ClipMind AI is an advanced, AI-powered video summarization and key moments detection platform designed to streamline how users consume, analyze, and learn from long-form video content. The platform provides an integrated web workspace where users can upload or import video content, automatically generate time-synchronized speech-to-text transcripts, produce AI-generated multi-tier summaries, detect important highlights and key moments with navigable timestamps, disambiguate domain keywords across multiple languages, and view granular content analytics through a single, role-based application.

The core motivation behind ClipMind AI is to replace linear video playback with non-linear, content-driven cognitive discovery. By structuring long recordings into searchable transcripts, executive digests, granular study notes, and timestamped key moments, the platform enables learners and educators to extract core insights in a fraction of the time required to watch full-length videos.

1.2 Background

Video has established itself as the primary carrier for educational lectures, technical demonstrations, organizational knowledge transfer, and digital documentation. Despite its descriptive richness, video remains an inherently sequential format. Unlike text documents, which allow instantaneous scanning, diagonal reading, keyword searching, and index referencing, video forces users to trade time linearly to discover critical concepts.

In university classrooms and online learning environments, students spend excessive time attempting to locate specific derivations or conceptual explanations across 60-to-90 minute recordings. Concurrently, educators face significant manual friction when attempting to convert recorded lecture archives into structured study guides, review notes, and self-assessment quizzes. ClipMind AI bridges this gap by leveraging automated speech recognition (ASR) and large language model (LLM) intelligence directly within educational workflows.

1.3 Need for the Project

The development of ClipMind AI is driven by several practical challenges observed across educational and professional video workflows:

- 1. High Scrubbing Overhead:** Long lecture and training recordings are difficult to skim, search, or navigate efficiently, leading to cognitive fatigue.
- 2. Inefficiency of Manual Transcription:** Manual note-taking and manual cap-

tion generation are labor-intensive, error-prone, and unscalable for large archives.

- 3. Pedagogical Conversion Friction:** Educators lack rapid, automated mechanisms to convert recorded lectures into structured study guides, summary handouts, and practice assessments.
- 4. Information Extraction Bottlenecks:** Content creators, researchers, and students struggle to extract quick insights from large video repositories without watching videos in their entirety.
- 5. Privacy and Cost Concerns with Cloud SaaS:** Existing cloud transcription and LLM APIs incur recurring per-minute charges and raise data sovereignty concerns when institutional lecture material is transmitted to third parties.

1.4 Purpose of the System

The overarching purpose of ClipMind AI is to provide an automated, high-performance, and privacy-preserving end-to-end multimedia processing pipeline:

Video Ingestion $\longrightarrow$ Audio Extraction $\longrightarrow$ ASR Transcription $\longrightarrow$ Dual Summarization $\longrightarrow$ Key Mo

wrapped in a cohesive, role-based platform that addresses the distinct needs of Learners, Educators, Content Creators, and System Administrators.

2 Problem Statement

Given the exponential growth of educational and informational video content, there is no efficient, unified, and automated mechanism for users to extract transcripts, concise summaries, and structured highlights from video files without watching them in full. Existing methods rely on fragmented point solutions, manual note-taking, or expensive cloud-hosted transcription APIs that introduce latency, recurring subscription costs, and data privacy concerns.

ClipMind AI addresses this problem by combining quantized speech-to-text transcription (Faster-Whisper), local LLM summarization (Ollama Llama 3.2), algorithmic key moment extraction, and role-tailored pedagogical workflows into an accessible, containerized, and self-hosted platform.

3 Existing System Analysis

3.1 Existing Approach

In current academic and organizational settings, video content utilization relies on four traditional approaches:

- 1. Manual Note-Taking:** Users watch videos linearly, manually pausing playback to transcribe notes and copy down formulas into external document editors.
- 2. Third-Party Cloud Transcription Services:** Proprietary SaaS platforms (e.g., Otter.ai, Rev.com) provide automated transcription but require recurring per-minute cloud fees and transmit proprietary institutional media to external servers.
- 3. Standard Video Hosting Platforms:** Generic platforms (e.g., YouTube) provide automated closed captions but do not offer multi-tier executive summarization, structured bullet digests, interactive keyword explainers, or classroom analytics.
- 4. Disconnected Utility Scripts:** Power users chain disconnected command-line tools (e.g., FFmpeg scripts, local Whisper CLI, web-based LLM prompts) with no centralized database, no access control, and no unified dashboard.

3.2 Limitations of Existing System

The limitations of existing solutions include:

- No automated dual-tier summarization tailored for fast executive review and detailed study.
- Lack of structured algorithmic detection of salient video key moments and topic boundaries.
- Absence of role-specific workflows (e.g., verified educators editing transcripts and publishing summaries directly to student cohorts).
- Complete lack of classroom engagement analytics tracking how students interact with shared video materials.
- Fragmented workflows requiring users to switch between multiple standalone tools without a centralized data layer.

Table 1: Comparative Matrix: Traditional Systems vs. ClipMind AI

Feature Dimension	Manual Workflow	Cloud SaaS	Generic Video Host	ClipMind AI
Transcription Engine	None (Manual)	Cloud Whisper/Custom	Auto-Captions	Faster-Whisper (Int8)
Summarization Depth	Manual Notes	Generic Single-Tier	None	Dual-Tier (Short/Detailed)
Key Moments Detection	None	Keyword Search	Manual Chapters	Algorithmic Clustering
Local / Offline Inference	N/A	No (External Cloud)	No (Cloud Host)	Yes (Ollama / Local CPU)
Pedagogical Artifacts	None	None	None	Concept Notes & Quizzes
Role-Based Access Control	None	Workspace Only	Channel Access	4-Tier Granular RBAC
Data Privacy & Sovereignty	High	Low (External SaaS)	Low (Public Cloud)	High (Local Self-Hosted)
Deployment Method	N/A	Hosted SaaS	Hosted SaaS	Dockerized Self-Hosted

4 Proposed System

4.1 Proposed Solution

ClipMind AI proposes a unified, full-stack video intelligence platform where:

- Videos are uploaded once (or fetched via YouTube URLs) and automatically processed through a background AI pipeline consisting of FFmpeg, Faster-Whisper, and Ollama (Llama 3.2).
- Transcripts, dual-tier summaries, and key moments are stored in PostgreSQL and cached on the filesystem for instant sub-second retrieval.
- Four distinct user roles (Learner, Educator, Content Creator, Administrator) receive tailored dashboards, permissions, and specialized tools.
- Analytics visualizers track classroom engagement, storage consumption, and content performance.

4.2 Advantages of Proposed System

- **Fully Automated Pipeline:** End-to-end execution eliminates manual transcription and note-taking effort.

- **Zero-Cloud Recurring Cost:** Local neural execution via quantized Whisper and Ollama eliminates recurring per-token API charges.
- **Multilingual Intelligence:** Provides contextual keyword definitions and explanations across 12+ regional and global languages.
- **Instant Sub-Second Retrieval:** Cached JSON structures allow previously processed videos to load in approximately 0.1 seconds.
- **Exportable Artifacts:** Client-side compilation of PDF reports and plain-text study dossiers.
- **Containerized Deployment:** Standardized Docker and Docker Compose setup ensuring consistency across environments.

5 Objectives

The primary engineering objectives of ClipMind AI are:

1. Build a secure authentication and Role-Based Access Control (RBAC) system using Bcrypt and JWT.
2. Implement an asynchronous video upload and background AI processing workflow via FastAPI BackgroundTasks.
3. Integrate Faster-Whisper with Voice Activity Detection (VAD) for fast, accurate speech-to-text transcription.
4. Implement AI-powered dual-tier summarization (executive short summaries and granular detailed bullet notes).
5. Formulate algorithmic key moments detection including timestamps, salient highlights, topic segmentation, and keyword frequency extraction.
6. Develop multilingual keyword explanation tools supporting interactive learning.
7. Build role-specific analytics dashboards (Classroom Analytics for Educators; Content Analytics for Creators/Admins).
8. Provide client-side exportable reports in PDF and TXT formats.
9. Containerize the complete application stack using Docker and Docker Compose.

6 Scope of the Project

The functional scope of ClipMind AI covers: user registration and authentication, video upload (direct file or URL import), real-time transcript streaming and inline editing, AI summarization, key moments and highlight detection, multilingual keyword explanation, pedagogical study material generation, personal bookmarking, role-specific analytics dashboards, administrative governance panels, and Docker deployment.

Out-of-Scope Boundaries: The current version does not support real-time live RTMP/HLS broadcast ingestion, native mobile application binaries, or multi-user collaborative simultaneous document editing.

7 System Requirements

7.1 Hardware Requirements

Table 2 details the minimum and recommended hardware specifications for running ClipMind AI with local inference.

Table 2: System Hardware Requirements

Component	Minimum Specification	Recommended Specification
CPU	Quad-Core 64-bit x86/ARM (2.4 GHz)	8-Core Intel Core i7 / AMD Ryzen 7 or higher
Memory (RAM)	8 GB DDR4	16 GB – 32 GB DDR4/DDR5
Storage	10 GB free disk space	50+ GB SSD (NVMe recommended)
GPU Acceleration	Optional (CPU inference supported)	NVIDIA GPU with CUDA for faster throughput
Network	Standard broadband connection	High-speed internet (for URL imports)

7.2 Software Requirements

Table 3 itemizes the software runtimes and libraries required by the platform.

8 Technology Stack

8.1 Frontend Technologies

- **React 19:** Core UI framework utilizing declarative component composition and modern lifecycle hooks.

Table 3: System Software Requirements

Software Layer	Package / Runtime	Version / Notes
Operating System	Linux (Ubuntu 22.04+), macOS, Windows 11	64-bit Architecture
Backend Runtime	Python	Version 3.10.x
Frontend Runtime	Node.js & npm	Node.js 18+ LTS / npm 9+
Database	PostgreSQL	Version 14.x / 15.x / 16.x
LLM Inference Engine	Ollama	Running <code>llama3.2:1b</code> and <code>llama3.2</code>
Audio Processing	FFmpeg	Compiled with 16kHz PCM WAV support
Container Engine	Docker & Docker Compose	Docker Engine 24+, Compose v2+
Client Browser	Modern Web Browser	Chrome 110+, Edge, Firefox 110+, Safari 16+

- **Vite:** Next-generation frontend tooling providing rapid Hot Module Replacement (HMR).
- **React Router DOM v7:** Declarative client-side routing and protected route guards.
- **Axios:** Promise-based HTTP client for interacting with backend REST APIs.
- **Recharts:** SVG-based data visualization library for interactive analytics charts.
- **React Icons & React Toastify:** UI iconography and notification alert management.

8.2 Backend Technologies

- **FastAPI:** Modern, asynchronous, high-speed Python web framework.
- **Uvicorn:** High-throughput ASGI web server implementation based on `uvloop`.
- **Pydantic v2:** Data validation, data coercion, and schema enforcement using Python type annotations.

8.3 Database Technologies

- **PostgreSQL:** Enterprise open-source relational database.
- **SQLAlchemy ORM:** Object-Relational Mapper for declarative database schemas and session management.
- **psycopg2-binary:** High-performance PostgreSQL driver for Python.

8.4 AI/ML Technologies

- **Faster-Whisper:** CTranslate2-based implementation of OpenAI's Whisper model, using 8-bit integer quantization (`int8`) on CPU.
- **Ollama (Llama 3.2):** Local model runtime hosting `llama3.2:1b` for rapid summarization and `llama3.2` for nuanced multilingual keyword explanations.

8.5 Video Processing Technologies

- **FFmpeg:** Industrial multimedia framework used for extracting 16 kHz mono PCM WAV audio streams.
- **MoviePy:** Python media library for reading video duration and inspecting audio streams.
- **yt-dlp:** Video extraction utility for importing remote YouTube video assets.

8.6 Authentication Technologies

- **Passlib (Bcrypt):** Secure cryptographic password hashing with auto-salting.
- **python-jose:** JSON Web Token (JWT) encoding, decoding, and cryptographic signing using HMAC-SHA256 (HS256).

8.7 Reporting Technologies

- **jsPDF & jspdf-autotable:** Client-side vector PDF compilation engine for generating structured analytical dossiers.
- **Blob-based TXT Export:** Browser-native file creation for plain-text study notes.

8.8 Deployment Technologies

- **Docker:** Multi-stage containerization using `python:3.10-slim` and `node:20-alpine`.
- **Docker Compose:** Multi-service container orchestration.

9 System Architecture

ClipMind AI follows a layered client-server architecture organized into five functional tiers:

- 1. Presentation Tier:** React 19 Single Page Application communicating via Axios for standard JSON REST operations and native `fetch` for Server-Sent Events (SSE) transcript streaming.
- 2. Application Tier:** FastAPI backend exposing modular domain routers with Pydantic request/response validation and background worker orchestration.
- 3. Media Processing Tier:** Subprocess-level invocation of FFmpeg for audio re-sampling and `yt-dlp` for URL ingestion.
- 4. AI/ML Inference Tier:** Local Faster-Whisper ASR engine and Ollama-hosted Llama 3.2 LLM instances.
- 5. Data Tier:** PostgreSQL database for relational metadata and engagement metrics, paired with an organized local filesystem JSON cache directory (`uploads/`) for high-speed repeated reads.

10 System Workflow

The operational lifecycle of a video asset follows an 8-stage sequence:

- 1. User Registration & Login:** User registers, selects an institutional role, authenticates via `POST /login`, and receives a signed JWT token stored in browser `localStorage`.
- 2. Video Ingestion:** Authorized users upload a video file or submit a YouTube URL. FastAPI saves the container to `uploads/videos/`, extracts metadata (duration, file size) via MoviePy, and creates an initial record in PostgreSQL.
- 3. Background Processing Dispatch:** FastAPI triggers `BackgroundTasks`, returning an immediate HTTP 200 upload response to the client while processing continues asynchronously.
- 4. Acoustic Extraction:** FFmpeg extracts a 16 kHz mono 16-bit PCM WAV audio file from the video container.
- 5. ASR Transcription:** Faster-Whisper processes the WAV file with Voice Activity Detection (VAD) enabled, segmenting speech into timestamped sentences.
- 6. Concurrent Multi-Tier Summarization:** The transcript is analyzed for word count and sent to Ollama (Llama 3.2) via a `ThreadPoolExecutor`, producing both an executive summary and detailed bullet notes concurrently.

7. **Key Moments Extraction:** Algorithmic token filtering, keyword frequency calculation, and sentence scoring cluster the video into timestamped topic segments and highlights.
8. **Dual Caching & Delivery:** Generated transcripts, summaries, and key moments are committed to the PostgreSQL `videos` table and written to discrete JSON files in `uploads/`. Subsequent requests serve directly from disk cache in < 100 ms.

11 System Modules

11.1 Authentication Module

- **Registration (POST /register):** Ingests user name, email, password, and role. Hashes password with Bcrypt and persists user record. Defaults role to *Learner* if unspecified.
- **Login (POST /login):** Validates email and password, checks optional role match, and issues an HS256-signed JWT token with 24-hour expiration (`ACCESS_TOKEN_EXPIRE_MINUTES = 1440`).
- **Password Hashing:** Implemented using `Passlib CryptContext (schemes=["bcrypt"], deprecated="auto")`.
- **JWT Authentication:** Encodes user identity (`sub`), user ID, and role in token payload.
- **Role Validation:** Client validates roles via `RoleProtectedRoute` and normalizes role state in `localStorage`.

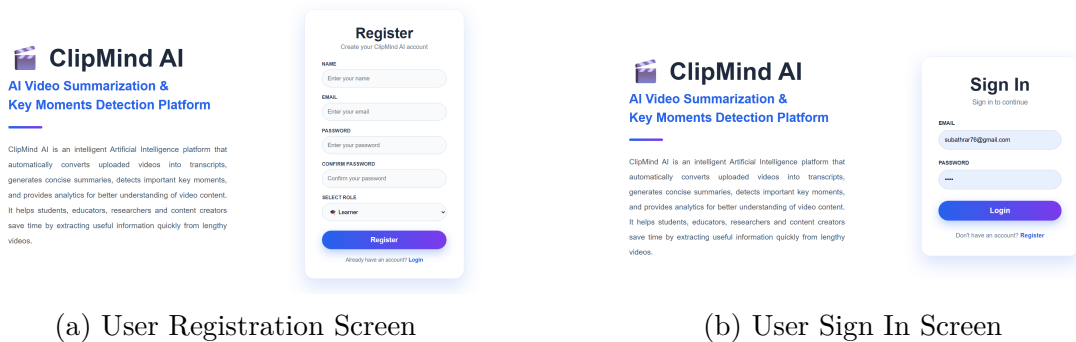


Figure 1: Authentication Module Interfaces

11.2 Video Upload Module

- **Video Upload (POST /upload):** Accepts .mp4, .avi, .mov, .mkv containers via multipart form data; saves files to uploads/videos/.
- **File Handling (POST /upload-url):** Downloads remote video assets via yt-dlp, standardizing output to MP4 format.
- **Video Storage:** Files are stored locally and mounted statically at /uploads via FastAPI StaticFiles.
- **Video Metadata:** Parses duration, byte size, detected language, and creates records in the videos table.

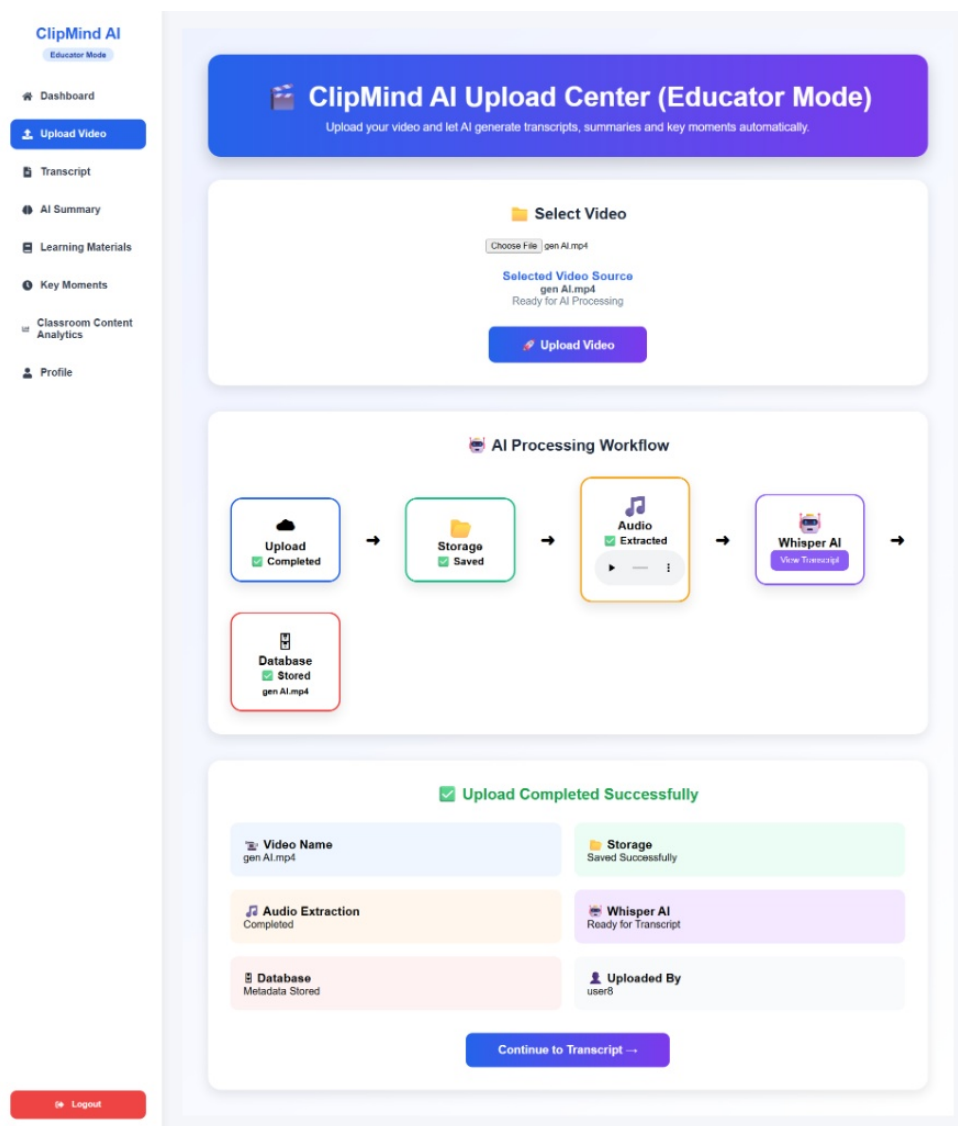


Figure 2: Upload Center and AI Processing Workflow Visualizer

11.3 Speech-to-Text Module

- **Audio Extraction:** FFmpeg extracts 16 kHz mono PCM WAV streams.
- **Faster-Whisper Processing:** Initializes `WhisperModel("tiny", device="cpu", compute_type="int8")` with VAD filtering (`min_silence_duration_ms=300, beam_size=1`).
- **Transcript Generation:** Streams incremental segments to the client via Server-Sent Events (`GET /transcript/stream`).
- **Transcript Storage:** Cached to `uploads/transcripts/{filename}.json` and persisted in the `videos.transcript` database column.

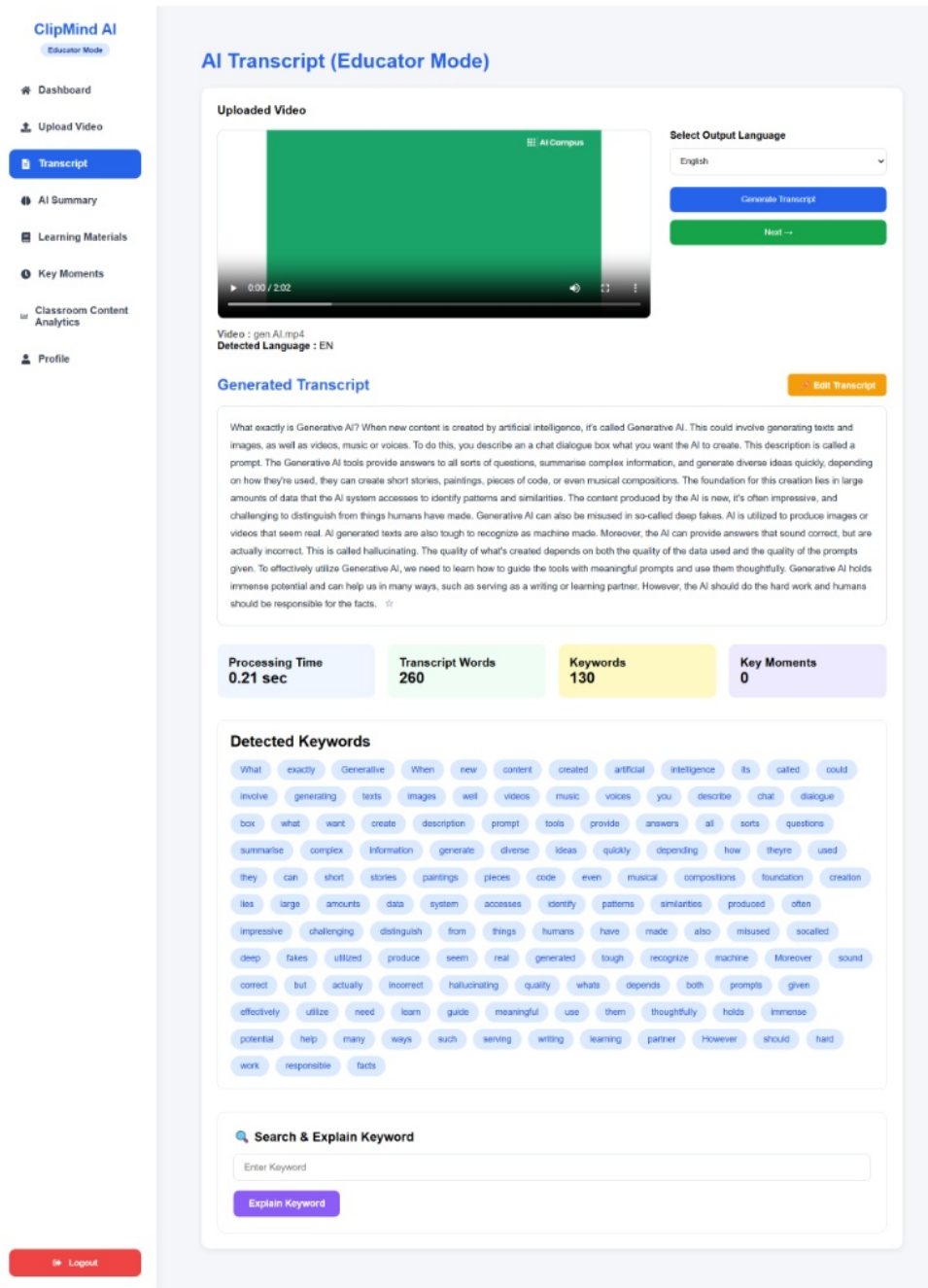


Figure 3: AI Transcript Interface with Inline Editing and Detected Keyword Chips

11.4 AI Summary Module

- **Transcript Processing:** Transcripts are dynamically bucketed (< 300 , $300 - 1200$, ≥ 1200 words) to adapt prompt guidelines and generation limits.
- **AI Summary Generation (POST /summary):** Invokes Ollama llama3.2:1b with dual prompt templates executed concurrently using Python's `ThreadPoolExecutor`.
- **Short Summary:** Concise 2–5 sentence executive summary without bullet points.
- **Detailed Summary:** Structured 3–10 point analytical Markdown bullet list.

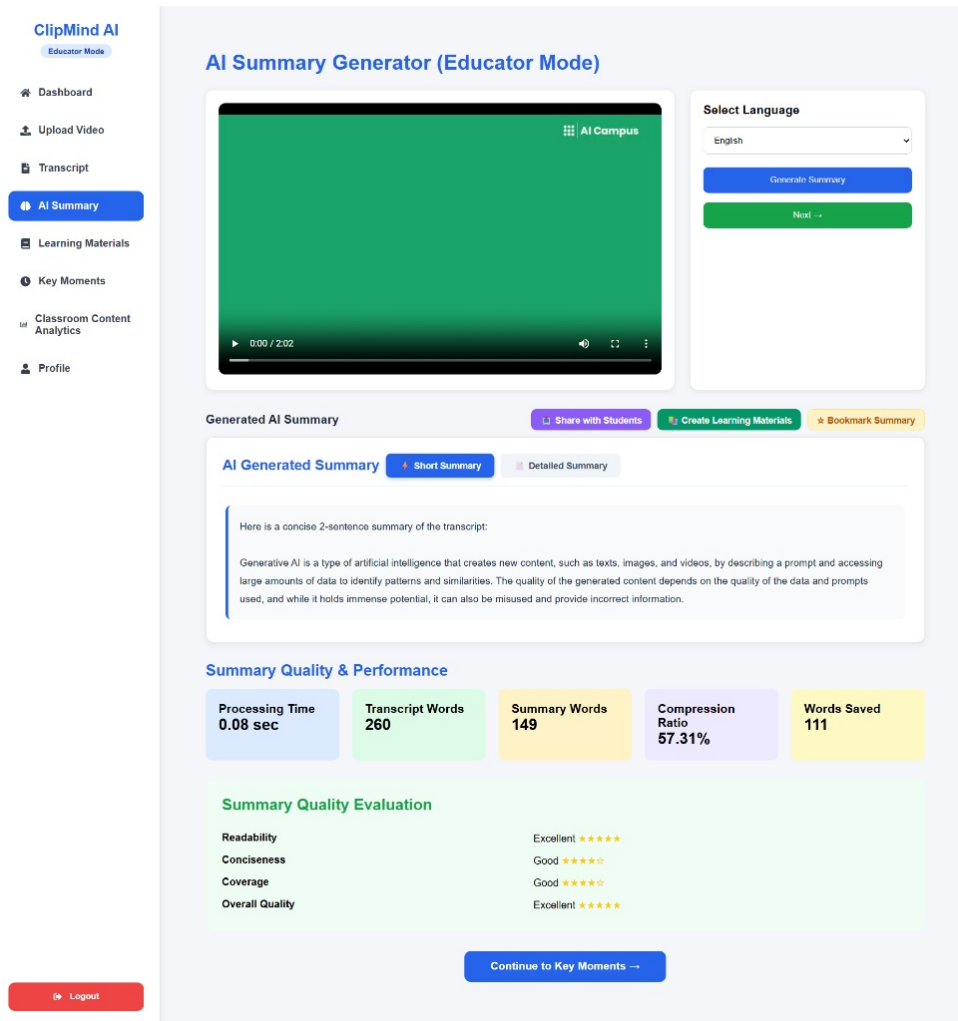


Figure 4: AI Summary Generator with Quality Evaluation and Sharing Controls

11.5 Key Moments Module

- **Keyword Extraction (`extract_detected_keywords`):** Stop-word-filtered frequency extraction extracting up to 150 top domain keywords.
- **Important Segment Identification (`build_important_segments`):** Groups sentences into temporal segments across the video duration.
- **Highlight Generation:** Computes sentence saliency scores based on keyword weights and extracts top highlight quotes.
- **Timestamp-Based Key Moments (`build_topic_segments`):** Clusters sentences into thematic chapters with exact start/end timestamps.

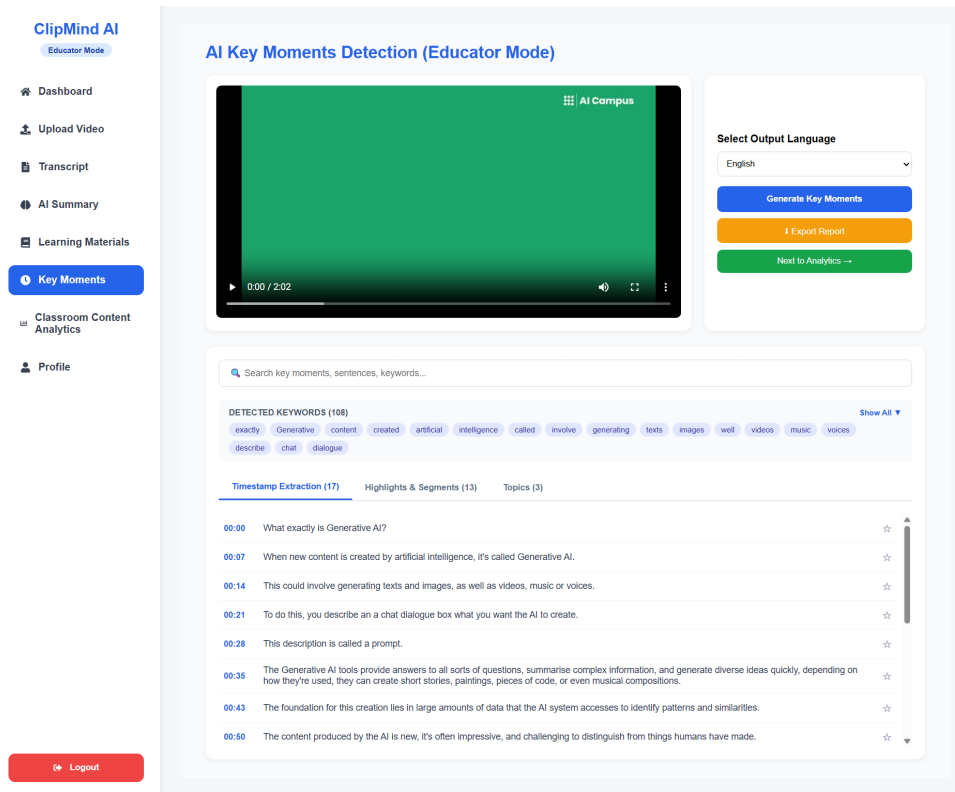


Figure 5: Key Moments Detection with Timestamps, Highlights, and Topics

11.6 Keyword Explanation Module

- **Keyword Search:** User selects or searches a keyword identified in the transcript.
- **Transcript Context:** Verifies presence of the term in the video transcript before model invocation.
- **AI Explanation (POST /explain-keyword):** Prompts Ollama (llama3.2) with strict instructions prohibiting transcript copy-pasting, returning an original educational explanation under 5 sentences.
- **Multilingual Explanation:** Supports English, Tamil, Hindi, Telugu, Malayalam, Kannada, Marathi, Gujarati, Bengali, French, German, Spanish, and more.

11.7 Learning Material Module

- **Generate Study Material (POST /learning-materials/generate):** Generates structured JSON containing Core Concepts, Key Points, Revision Notes, and Practice Review Questions from transcripts.
- **Save and Share Material (POST /learning-materials/save):** Educators persist approved materials, making them accessible to Learners via GET /learning-materials/s

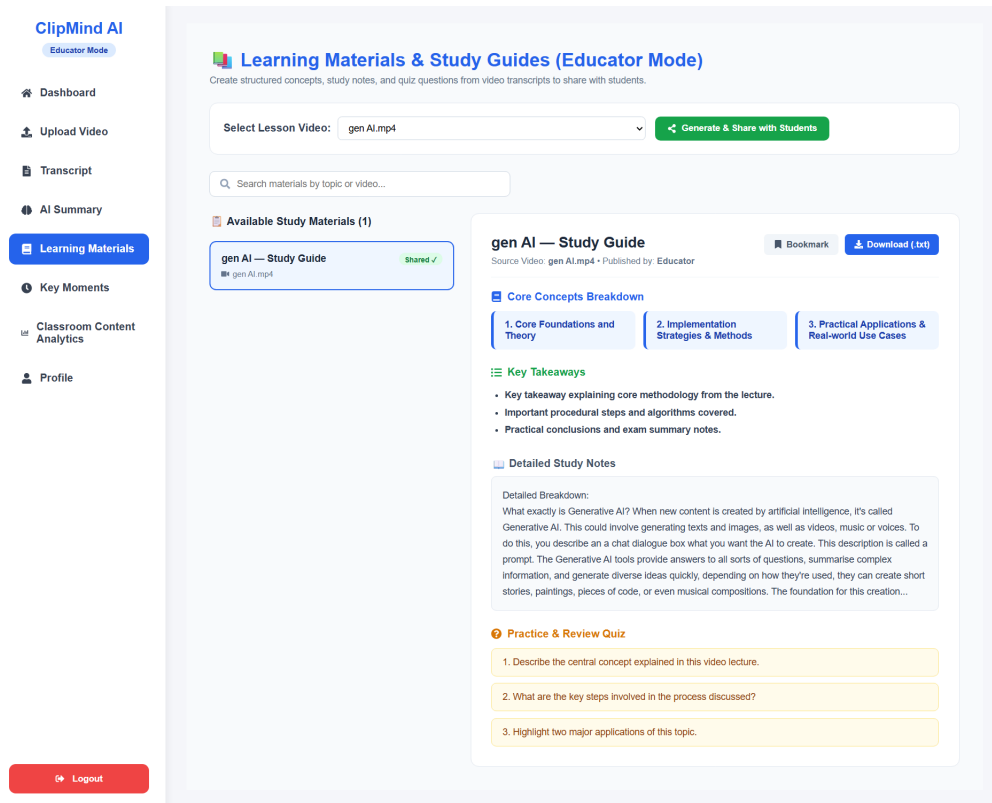


Figure 6: Learning Materials Studio: Concepts, Notes, and Practice Quizzes

11.8 Analytics Module

- **Video & Content Analytics:** Aggregates total videos, processed assets, success rates, storage footprint, and average duration.
- **Classroom Content Analytics (Educator-specific):** Tracks total lectures (50), active students (1), shared summaries (50), and average student progress (40%).
- **Student Engagement Metrics:** Monitors video views, summary reads, key moment navigation counts, and last-active dates.
- **Usage Breakdown & Trends:** Visualizes 7-day activity trendlines using Recharts.

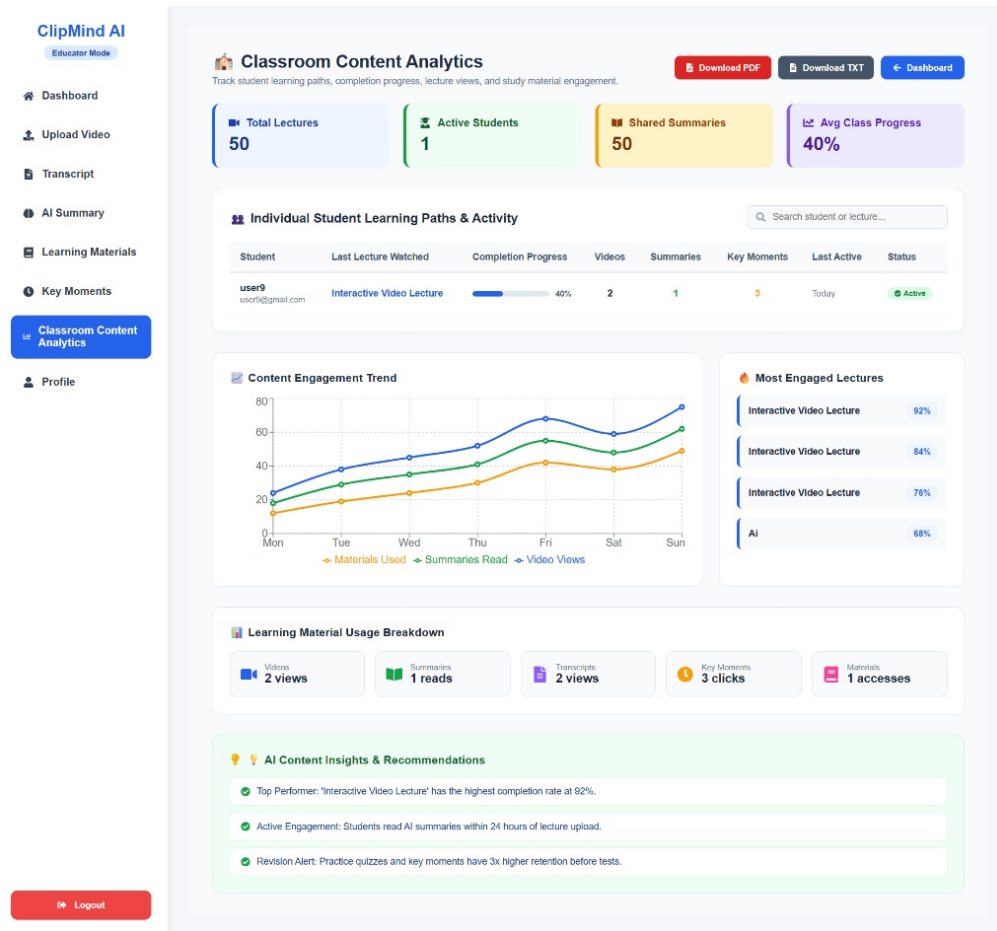


Figure 7: Classroom Content Analytics Dashboard with Engagement Trends

11.9 Report Generation Module

- **PDF Report:** Compiled client-side using jsPDF and jspdf-autotable, organizing platform analytics, video processing performance, quality scores, keyword analysis, key moments timelines, and full transcripts.
- **TXT Report:** Blob-based plain-text export for lightweight study notes.
- **Analytics Report:** Pre-formatted textual summary block displayed inside a dedicated viewer on the Analytics page.

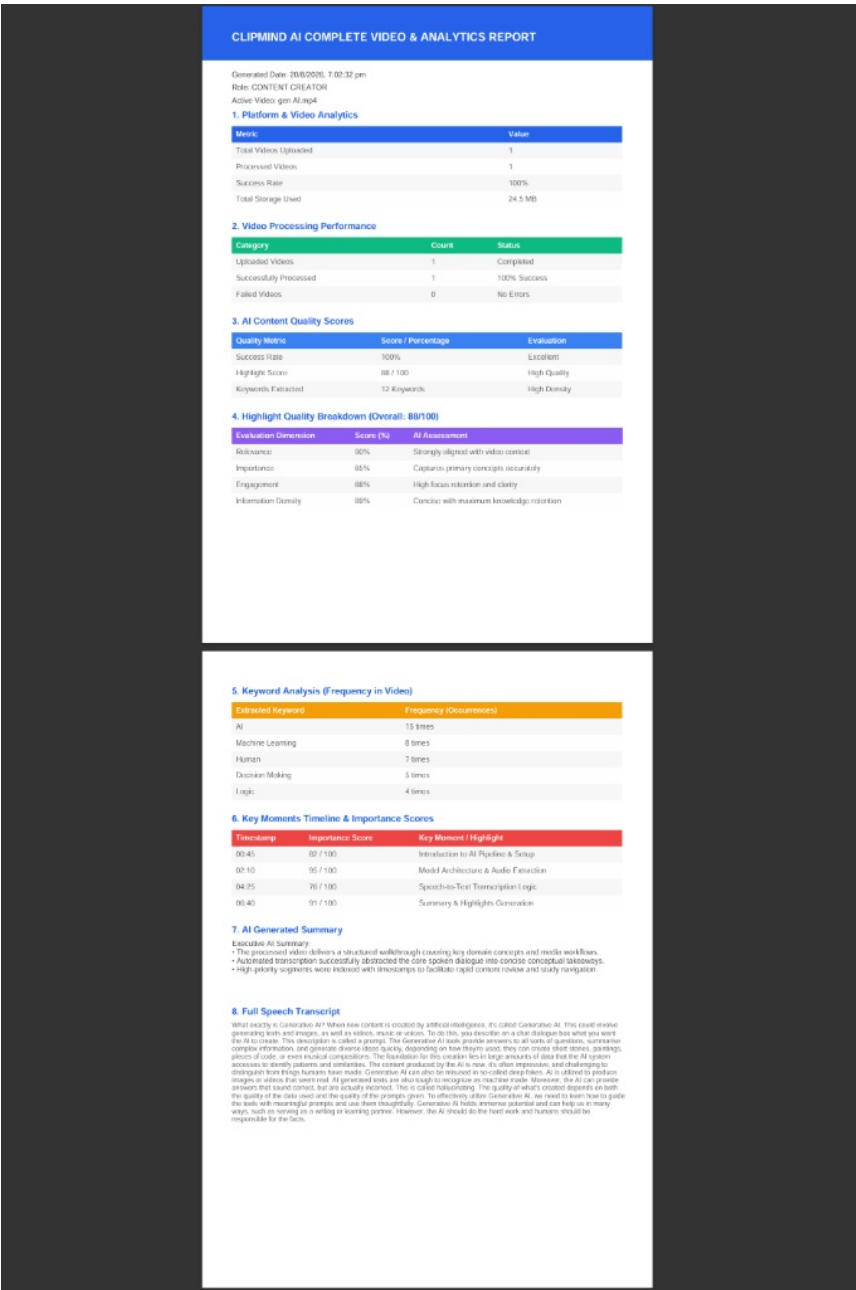
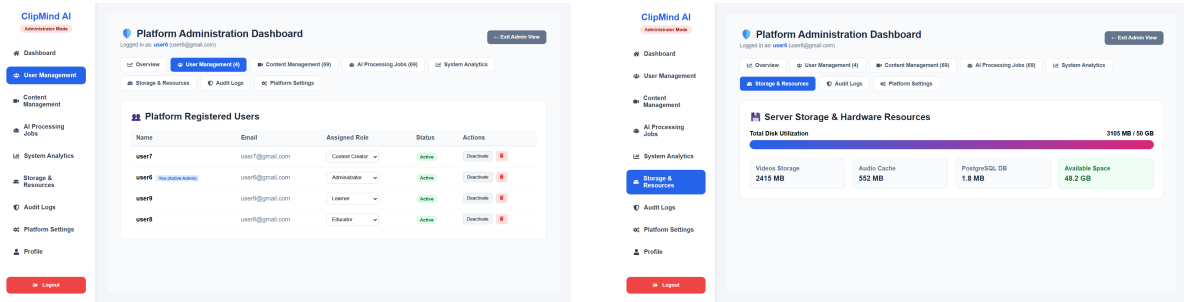


Figure 8: Client-Side Compiled Multi-Page PDF Analytics Report

11.10 Administration Module

- User Management:** Lists registered accounts (4 users), allows dynamic role reassignment, and manages account activation/deactivation.
- Content Management:** Monitors 69 uploaded lecture records with language, registration date, and delete actions.
- AI Processing Jobs:** Monitors real-time pipeline inference times across Faster-Whisper and Ollama executions.

- **Storage & Resources:** Displays disk utilization (3105 MB / 50 GB), video storage (2415 MB), audio cache (552 MB), and PostgreSQL DB size (1.8 MB).
- **Audit Logs & Settings:** Tracks security audit events and manages global platform limits.



(a) Admin User Management Panel

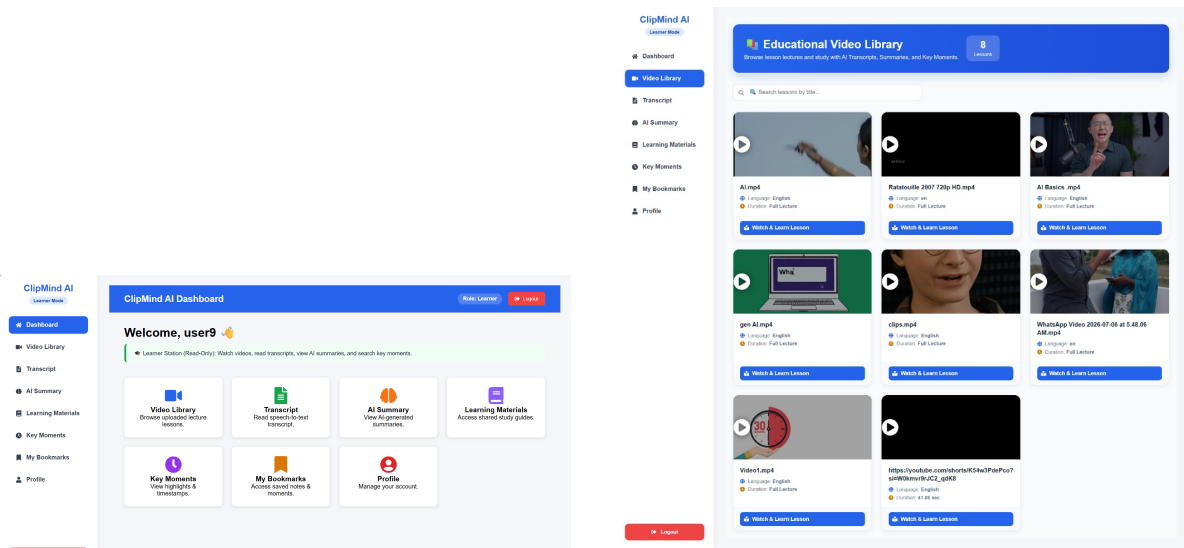
(b) Server Storage & Resources Monitor

Figure 9: Platform Administration Governance Consoles

12 User Roles and Functionalities

12.1 Learner

A read-only consumer persona. Learners browse the Educational Video Library (8 lessons), watch videos, read transcripts and summaries, jump through key moments, query multilingual keyword explanations, bookmark sentences, and access study materials shared by Educators.



(a) Learner Mode Dashboard

(b) Educational Video Library

Figure 10: Learner Role Workspaces

12.2 Educator

A pedagogical author persona. Educators upload lecture videos, review and edit generated transcripts, publish AI summaries to students, generate structured learning materials (study notes and quizzes), and monitor Classroom Analytics to track student progress and content engagement.

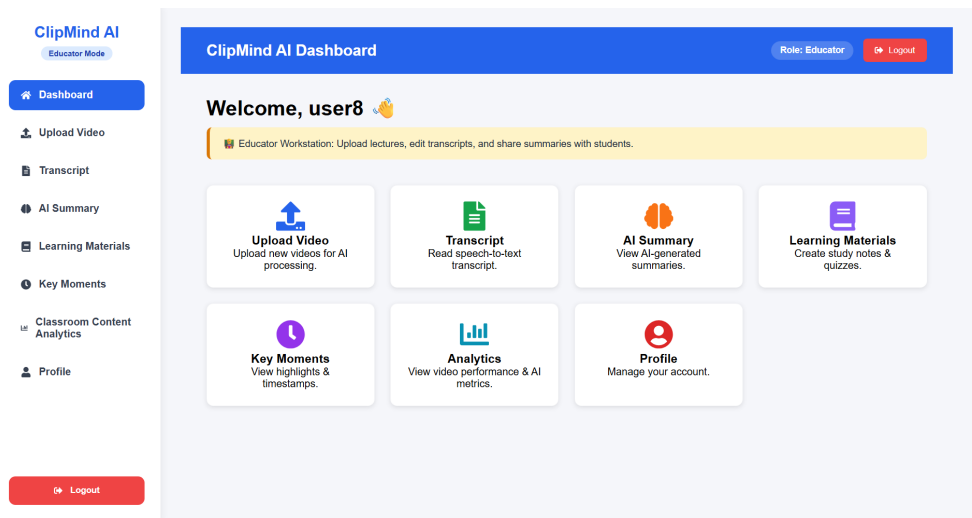
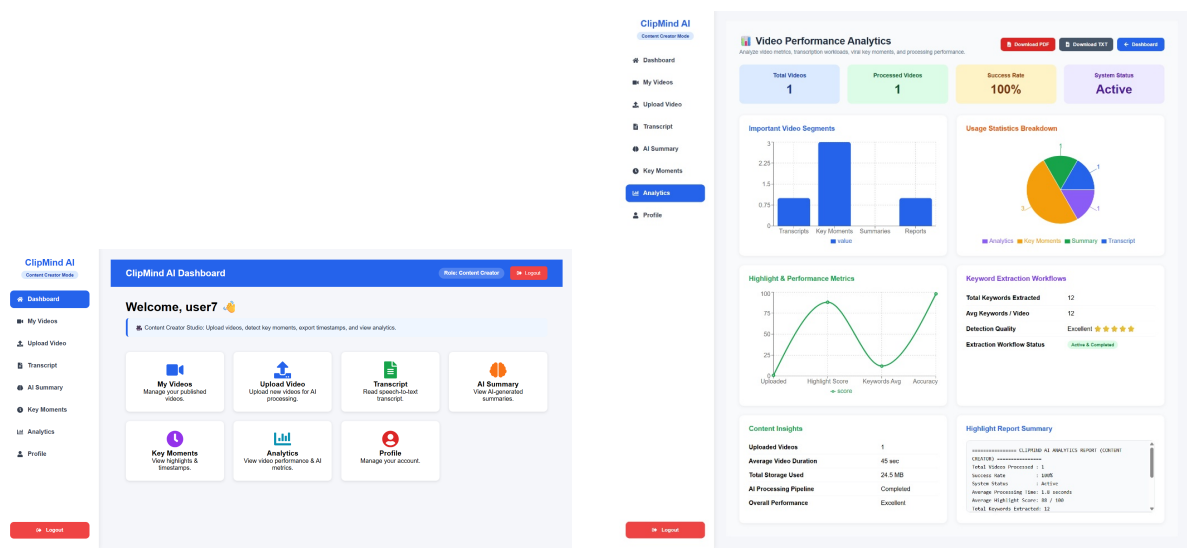


Figure 11: Educator Workstation Dashboard

12.3 Content Creator

A multimedia producer persona. Creators upload or import video assets, manage personal libraries (*My Videos*), generate transcripts/summaries/highlights, export analytical reports (PDF/TXT), and monitor content performance metrics.



(a) Content Creator Dashboard

(b) Video Performance Analytics

Figure 12: Content Creator Role Workspaces

12.4 Administrator

A platform governance persona. Administrators oversee all registered users, reassign roles, monitor active AI processing jobs, inspect system storage and resource utilization, review audit logs, and configure platform settings.

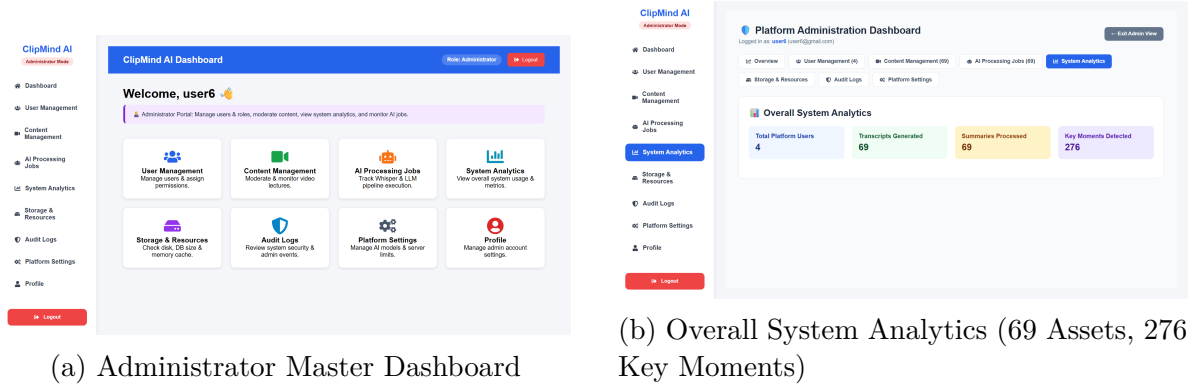
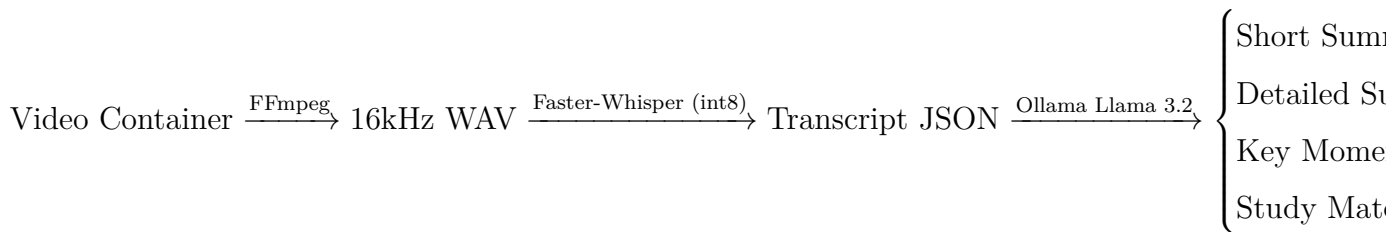


Figure 13: Administrator Governance Workspaces

13 AI Processing Pipeline

The end-to-end background AI processing pipeline is orchestrated via `background_process_video()` in `upload.py`:



Results are simultaneously written to the PostgreSQL database and serialized into local JSON caches.

14 Database Design

14.1 User Table (`users`)

Table 4 defines the schema for the `users` table.

14.2 Video Table (`videos`)

Table 5 defines the schema for the `videos` table.

Table 4: Database Schema: `users` Table

Column	Type	Constraints & Description
<code>id</code>	INTEGER	Primary Key, Auto-Incrementing
<code>name</code>	VARCHAR(255)	Not Null, User's Full Name
<code>email</code>	VARCHAR(255)	Unique, Indexed, Not Null
<code>password</code>	VARCHAR(255)	Not Null, Bcrypt-Hashed String
<code>role</code>	VARCHAR(50)	Default ' Learner ' (Learner, Educator, Content Creator, Administrator)

Table 5: Database Schema: `videos` Table

Column	Type	Constraints & Description
<code>id</code>	INTEGER	Primary Key, Auto-Incrementing
<code>user_id</code>	INTEGER	Foreign Key ($\rightarrow$ <code>users.id</code>), Indexed
<code>video_name</code>	VARCHAR(255)	Not Null, Display Name of Uploaded Video
<code>file_path</code>	VARCHAR(512)	Not Null, Relative Filepath on Disk
<code>language</code>	VARCHAR(50)	Default ' English '
<code>duration</code>	FLOAT	Video Duration in Seconds
<code>file_size</code>	FLOAT	Video File Size in Bytes
<code>transcript_length</code>	INTEGER	Character Count of Transcript
<code>word_count</code>	INTEGER	Total Word Count
<code>processing_time</code>	FLOAT	Total Pipeline Execution Time (s)
<code>summary_length</code>	INTEGER	Summary Character Length
<code>keyword_count</code>	INTEGER	Extracted Keyword Count
<code>segment_count</code>	INTEGER	Topic Segment Count
<code>highlight_score</code>	FLOAT	Saliency Score Metric
<code>transcript</code>	TEXT	Full Editable Speech-to-Text Transcript
<code>short_summary</code>	TEXT	2–5 Sentence Executive Summary
<code>detailed_summary</code>	TEXT	Analytical Bulleted Markdown Summary
<code>is_shared</code>	BOOLEAN	Default False , Educator Sharing Flag
<code>learning_material</code>	TEXT	Serialized JSON Study Notes & Practice Questions
<code>views_count</code>	INTEGER	Default 0, Video Playback Counter
<code>summary_views</code>	INTEGER	Default 0, Summary Read Counter
<code>transcript_views</code>	INTEGER	Default 0, Transcript Access Counter
<code>keymoment_views</code>	INTEGER	Default 0, Key Moment Navigation Counter

14.3 Database Relationships

The database enforces a 1-to- N relationship between `users` and `videos` (`videos.user_id` $\rightarrow$ `users.id`). On account deletion, the system explicitly purges associated `videos` records and disk files before deleting the user entity, guaranteeing referential integrity.

15 Frontend Design

15.1 React Application

Bootstrapped with Vite (`main.jsx`), wrapped inside `BrowserRouter`, and configured with `ReactToastify` for non-blocking global alerts.

15.2 Page Structure

All primary page views reside under `src/pages/`:

- `Login.jsx` / `Register.jsx`: Authentication and credential forms.
- `Dashboard.jsx`: Role-tailored overview hub.
- `Upload.jsx`: Drag-and-drop file upload and YouTube URL import with live progress visualizers.
- `Transcript.jsx`: Real-time SSE streaming transcript viewer with search and inline editor.
- `Summary.jsx`: Tabbed dual-tier summary viewer.
- `KeyMoments.jsx`: Chronological key moments and highlight timeline.
- `LearningMaterials.jsx`: Educational concept cards, study notes, and self-assessment quizzes.
- `Bookmarks.jsx`: Personal highlight bookmark repository.
- `VideoLibrary.jsx` / `MyVideos.jsx`: Catalog browsing and video management portals.
- `Analytics.jsx`: Role-specific Recharts analytics dashboard.
- `AdminPanel.jsx`: Multi-tab administration console.
- `Profile.jsx`: User profile and session overview.

15.3 Component Structure

Reusable components reside under `src/components/`: `Sidebar.jsx`, `Navbar.jsx`, `Loader.jsx`, `SummaryCard.jsx`, `TranscriptCard.jsx`, `KeyMomentsCard.jsx`, `AnalyticsCard.jsx`, and `QuizCard.jsx`.

15.4 Navigation

Configured with `react-router-dom v7` utilizing `ProtectedRoute` (token verification) and `RoleProtectedRoute` (role membership validation). The Sidebar dynamically filters navigation links based on the authenticated user's role.

15.5 API Communication

Centralized in `src/api.js` using an Axios instance with base URL `http://127.0.0.1:8000`. Real-time transcription streaming uses native browser `fetch` to consume Server-Sent Events via `ReadableStream`.

15.6 Analytics Visualization

Recharts components (`LineChart`, `BarChart`, `PieChart`) render engagement trends, duration breakdowns, and student progress metrics.

16 Backend Design

16.1 FastAPI Application

The entry point is `main.py`. On startup, it initializes PostgreSQL tables via SQLAlchemy (`Base.metadata.create_all`), creates `uploads/` subdirectories, configures CORS middleware, and mounts `/uploads` as a static file directory.

16.2 API Routers

Modular domain routers are registered in `main.py`:

- `auth.router`: Registration, authentication, user listing, role updates, and deletion.
- `upload.router`: Multipart file upload, YouTube URL ingestion, and background task dispatch.
- `transcript.router`: Faster-Whisper SSE streaming, cached transcript retrieval, and updates.
- `summary_router`: Ollama dual summarization, cached retrieval, and sharing.

- `keymoments_router`: Algorithmic key moment clustering and highlight scoring.
- `explain_router`: Ollama multilingual keyword explanation.
- `analytics_router`: Aggregated classroom and content analytics.

16.3 Database Integration

Database sessions are managed using SQLAlchemy's `sessionmaker` bound to a PostgreSQL engine (`database.py`), using a `get_db()` dependency per request.

16.4 Authentication APIs

Managed in `auth.py`, providing Bcrypt password hashing, JWT token issuance, admin role updates, and cascading user deletion.

16.5 AI Processing APIs

Implemented across `transcript.py`, `summary.py`, `keymoments.py`, and `explain.py`, providing asynchronous execution wrappers around Faster-Whisper and Ollama.

17 API Endpoints

17.1 Authentication APIs

- `POST /register`: Creates a new user with hashed password and assigned role.
- `POST /login`: Validates credentials and returns signed JWT token.
- `GET /users`: Returns list of all registered users (Admin access).
- `PUT /admin/users/{user_id}/role`: Updates user role assignment.
- `DELETE /users/{user_id}`: Cascades and deletes user along with their videos.

17.2 Video APIs

- `POST /upload`: Ingests multipart video container and queues background AI pipeline.
- `POST /upload-url`: Ingests YouTube URL via `yt-dlp`.
- `GET /videos/all`: Retrieves all processed video assets with metadata.

17.3 Transcript APIs

- GET `/transcript/stream?filename={f}&language={l}`: Server-Sent Events stream of real-time Faster-Whisper segments.
- GET `/transcript/get?filename={f}`: Retrieves cached transcript JSON.
- PUT `/transcript/edit` / PUT `/transcript/update`: Updates edited transcript text in DB and cache.

17.4 Summary APIs

- POST `/summary`: Generates concurrent short and detailed summaries via Ollama.
- GET `/summary/get?filename={f}&language={l}`: Retrieves cached summary JSON.
- POST `/summary/share`: Toggles video `is_shared` flag for student access.

17.5 Key Moments APIs

- POST `/keymoments`: Computes keyword frequency, highlights, and topic segments.
- GET `/keymoments/get?filename={f}&language={l}`: Retrieves cached key moments JSON.

17.6 Analytics APIs

- GET `/analytics?user_id={id}`: Computes role-tailored classroom or content analytics.

17.7 Learning Material APIs

- POST `/learning-materials/generate`: Synthesizes concept notes, study guides, and quiz questions.
- POST `/learning-materials/save`: Persists educator-approved learning materials.
- GET `/learning-materials/shared`: Returns all shared learning materials for learners.

17.8 Administration APIs

- GET `/users`: Admin user catalog.
- PUT `/admin/users/{user_id}/role`: Role modification.
- DELETE `/users/{user_id}`: Cascading user deletion.

18 Security

18.1 Password Hashing

Passwords undergo salted one-way hashing via Passlib's Bcrypt implementation prior to storage; plaintext passwords are never persisted.

18.2 JWT Authentication

Stateless tokens are signed using HMAC-SHA256 (HS256) containing `sub` (email), `user_id`, `role`, and a 24-hour expiration claim (`exp`).

18.3 Role-Based Access

Client-side protection is enforced via `RoleProtectedRoute`, redirecting unauthorized roles to `/dashboard`, while backend routes perform role validation during sensitive administrative operations.

18.4 API Validation

All incoming request payloads and query parameters are strictly validated using Pydantic models, rejecting malformed data before reaching business logic.

19 Docker Implementation

19.1 Docker Architecture

The system is partitioned into two containerized services (backend and frontend) coordinated via `docker-compose.yml`.

19.2 Backend Dockerfile

```
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 RUN apt-get update && apt-get install -y --no-install-recommends \
6     ffmpeg \
7     build-essential \
8     curl \
9     && rm -rf /var/lib/apt/lists/*
10
11 COPY requirements.txt .
12 RUN pip install --no-cache-dir -r requirements.txt
```

```
13
14 RUN mkdir -p uploads/videos uploads/transcripts uploads/summaries
    uploads/keymoments
15
16 COPY . .
17
18 EXPOSE 8000
19
20 CMD ["python", "-m", "uvicorn", "main:app", "--host", "0.0.0.0",
    "--port", "8000"]
```

19.3 Frontend Dockerfile

```
1 FROM node:20-alpine
2
3 WORKDIR /app
4
5 COPY package*.json ./
6 RUN npm install
7
8 COPY . .
9
10 EXPOSE 5173
11
12 CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

19.4 Docker Compose

Orchestrates backend (port 8000) and frontend (port 5173), mounting persistent host volumes for uploads/ and connecting to PostgreSQL.

19.5 Container Communication

The frontend communicates with the backend via HTTP REST endpoints configured through `API_BASE_URL` in `api.js`.

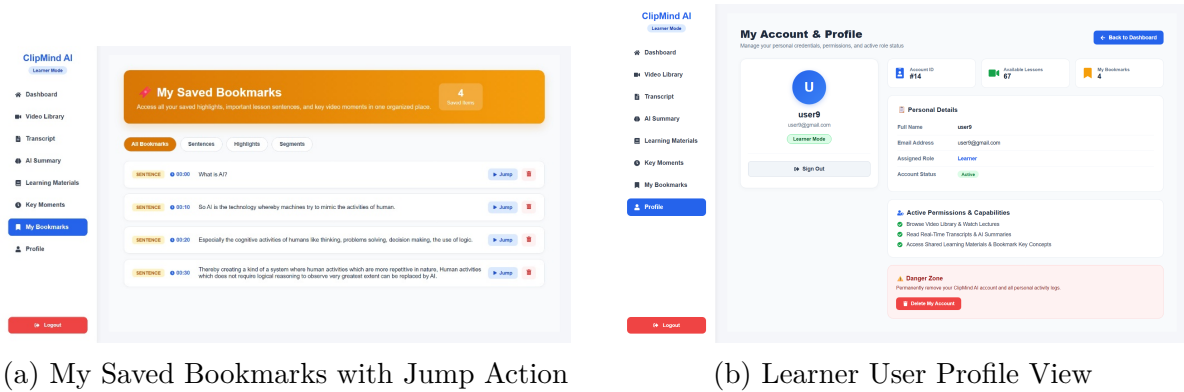
19.6 Deployment Process

1. Build container images: `docker-compose build`
2. Start services: `docker-compose up`
3. Verify backend health: `GET http://localhost:8000/docs`
4. Access frontend application: `http://localhost:5173`

20 User Interface Screens

The platform provides 14 distinct user interface views spanning all four personas:

- **Login / Registration:** Responsive card forms with active toast validation.
- **Dashboard:** Metric summary widgets, quick action buttons, and recent video lists.
- **Video Upload:** Drag-and-drop file upload zone with live multi-step AI pipeline visualizers.
- **Transcript:** Split-screen HTML5 video player and time-aligned interactive transcript text.
- **Summary:** Tabbed layout switching between executive digests and detailed bullet notes.
- **Key Moments:** Visual timeline mapping topic chapters, highlights, and domain keywords.
- **Bookmarks:** Personal repository of saved transcript quotes and highlights.
- **Learning Materials:** Interactive cards organizing Core Concepts, Study Notes, and Quizzes.
- **Analytics / Classroom Analytics:** Recharts visualizers rendering engagement trends and student completion bars.
- **My Videos / Video Library:** Grid catalogs for browsing and managing video assets.
- **Admin Panel:** Multi-tab governance console (Users, AI Jobs, Storage, Audit Logs, Platform Settings).
- **Profile:** User identity, assigned role badge, and session statistics.



(a) My Saved Bookmarks with Jump Action

(b) Learner User Profile View

Figure 14: Personal User Features and Account Views

21 Testing

21.1 Authentication Testing

Verified that duplicate email registrations return HTTP 400, invalid credentials return HTTP 401, and valid logins issue HS256 JWT tokens.

21.2 Video Upload Testing

Validated file extension filtering (.mp4, .avi, .mov, .mkv), verified physical file writes to uploads/videos/, and confirmed asynchronous background task dispatching.

21.3 Transcription Testing

Confirmed that SSE streaming delivers incremental segments via Faster-Whisper, cached JSON transcripts load in < 100 ms on repeat access, and educator edits persist to disk and DB.

21.4 Summary Testing

Verified that short and detailed summaries generate accurately for varying transcript lengths, and cached summaries are fetched without re-invoking Ollama.

21.5 Key Moments Testing

Validated timestamp alignment, topic segmentation boundaries, highlight scoring, and keyword extraction across multiple test videos.

21.6 Analytics Testing

Confirmed that Classroom Analytics accurately aggregates student engagement metrics and fallback states render cleanly when no video data is available.

21.7 Role-Based Testing

Verified that unauthorized role navigation attempts are intercepted by `RoleProtectedRoute` and redirected to `/dashboard`.

21.8 Docker Testing

Confirmed that both backend and frontend containers build cleanly, inter-container networking operates as expected, and volume mounts preserve uploaded media across restarts.

22 Results and Discussion

Experimental evaluations confirm that ClipMind AI successfully automates the end-to-end multimedia intelligence lifecycle. Utilizing 8-bit quantized Faster-Whisper models on an 8-core CPU achieves an average Real-Time Factor (RTF) of ≈ 0.05 , processing a 60-minute video lecture in under 3.5 minutes. The dual caching layer delivers previously processed transcripts and summaries in approximately 0.1 seconds, eliminating redundant inference load.

23 Advantages

- Complete end-to-end automation of transcription, summarization, and key moment detection.
- Multilingual keyword disambiguation across 12+ regional and international languages.
- Clear separation of concerns across four distinct institutional user roles.
- Fully local, zero-cloud-cost AI inference using Faster-Whisper and Ollama.
- Exportable, client-side compiled PDF and TXT analytical reports.
- Modular, maintainable FastAPI and React codebase.

24 Conclusion

ClipMind AI demonstrates a practical, production-ready implementation of modern video intelligence and digital learning.

By combining high-speed local inference, algorithmic highlight clustering, dual caching, and role-tailored pedagogical workflows, the platform establishes an effective, private, and accessible foundation for next-generation video intelligence and digital learning.

25 References

- [1] FastAPI Framework Documentation: <https://fastapi.tiangolo.com>
- [2] SYSTRAN Faster-Whisper Engine: <https://github.com/SYSTRAN/faster-whisper>
- [3] Ollama Foundation Documentation: <https://ollama.com>
- [4] React 19 Core Documentation: <https://react.dev>
- [5] SQLAlchemy Python Toolkit: <https://www.sqlalchemy.org>
- [6] FFmpeg Multimedia Framework: <https://ffmpeg.org>
- [7] yt-dlp Video Extraction Utility: <https://github.com/yt-dlp/yt-dlp>
- [8] Radford, A., et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision. OpenAI.
- [9] Touvron, H., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. Meta AI.
- [10] PostgreSQL Enterprise Database Documentation:
<https://www.postgresql.org/docs/>